Is the model accurate?

- **Every test score is a claim:** this is how the model will do on data it has not seen
- **The claim is easy to get wrong on climate data,** and the mistake makes the model look better than it is
- **Four main steps:** establish a baseline, use a split that matches the application, chose settings on validation set (or use cross-validation on the training set), and test the final model on the test set

A baseline

climatology, persistence, or the mean. No score means anything on its own

↓

A split that matches the application

many splits, not one, and folds that respect space and time

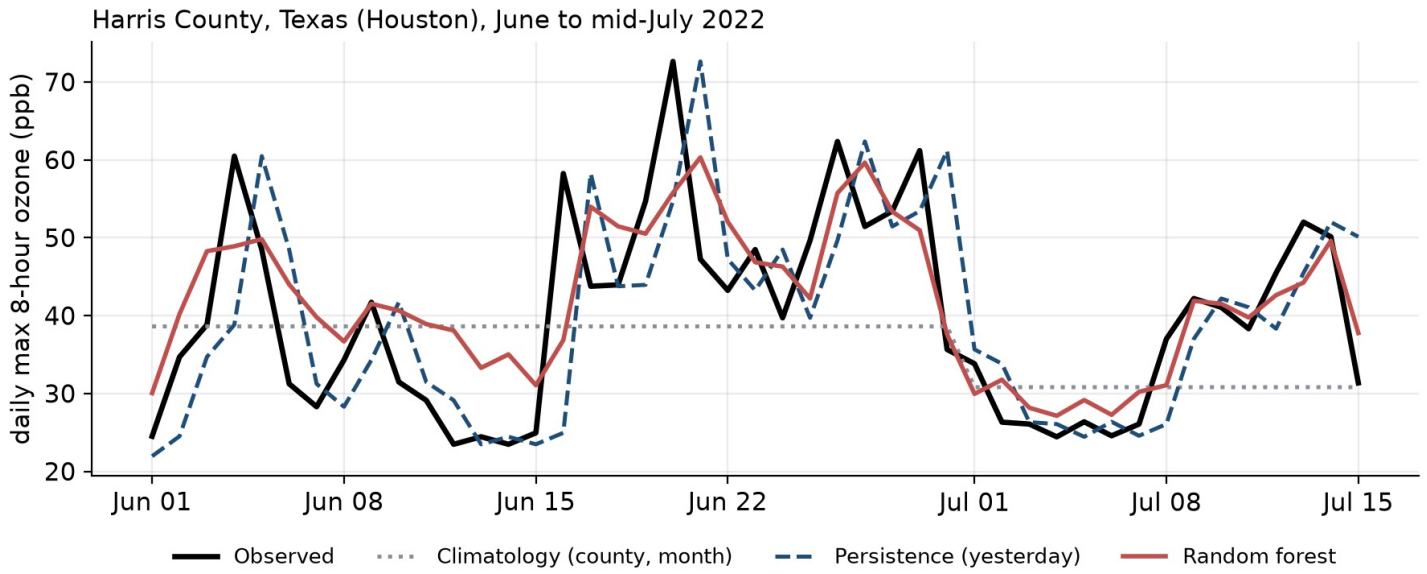
↓

Hyperparameters (model settings) chosen with cross-validation

validation data or cross-validation chooses them; the test set is scored once

Following the Week 6 lecture notes and Géron, Hands-On Machine Learning, 3rd ed., Ch. 2

Always compare against a baseline

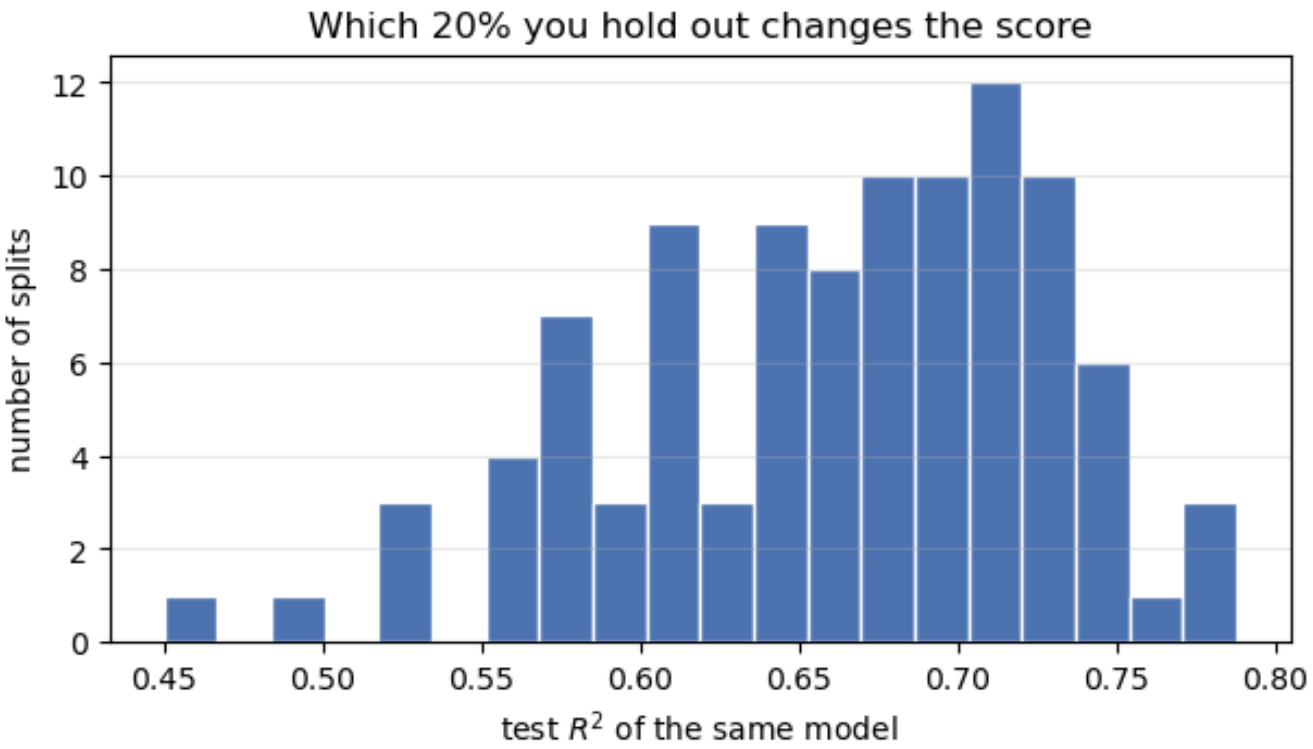


2022, 47,300 county-days	RMSE (ppb)	R^2
Climatology	8.4	0.44
Persistence	7.9	0.51
Random forest	5.9	0.73

- **Climatology:** the mean for that county and month, from the training years
- **Persistence:** yesterday's value. Hard to beat, because weather carries over from day to day
- **The forest beats both:** a quarter less error than persistence, from yesterday's ozone plus today's weather. That gap is the reason to build the model
- **A skill score without its baseline** is not a result

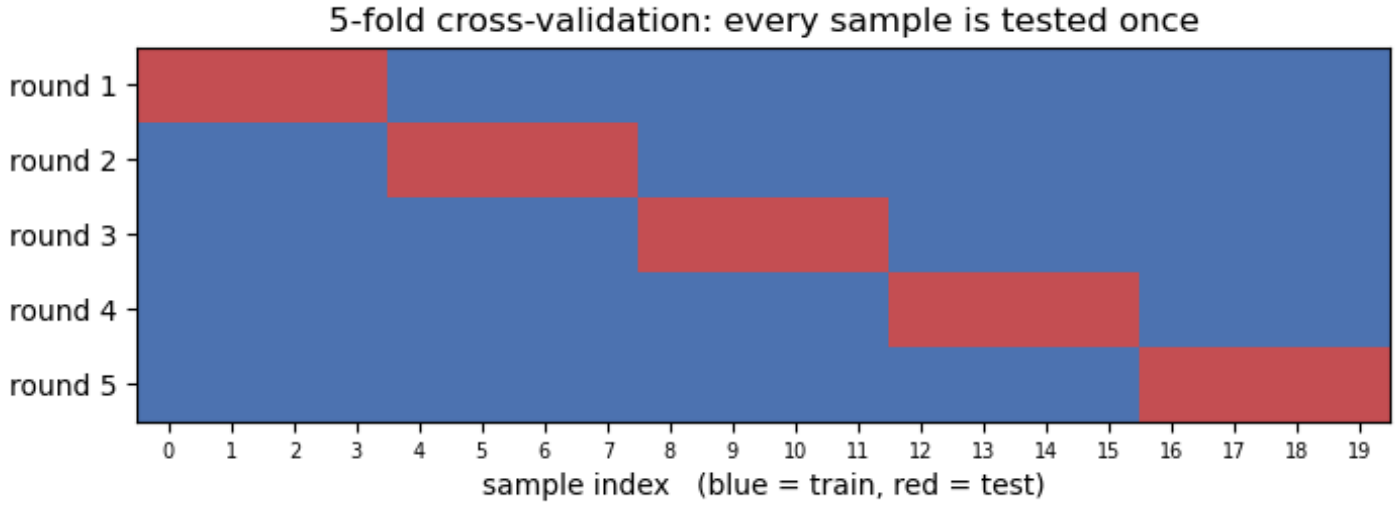
EPA Air Quality System daily maximum 8-hour ozone, county means, 172 counties in 50 states, 2019 to 2023. Forest trained on 2019 to 2021 with today's maximum temperature, wind, humidity, day of year, location, and yesterday's ozone

One hold-out score is one noisy number



- **Same model, same 300 samples, 100 random 80/20 splits**
- **Test R^2 runs from 0.45 to 0.79.** Which 60 samples you held out decided the score
- **A single hold-out is one draw from this histogram,** and you do not know where in it you landed
- **Small data sets make it worse,** and station records, field campaigns, and model ensembles are small

K-fold cross-validation



- **Cut the data into k folds.** Hold out each in turn, fit on the rest, score on the held-out fold
- **Every sample is tested once** and trained on k − 1 times
- **Report the mean and the spread** of the k scores, not one number
- **k = 5 or 10 is standard.**

What it buys: all the data tested, most of the data trained on in every round, and a spread.
What it costs: k fits instead of one.

- **Cross validation i**s a way to estimate how well a model will do on data it has not seen, by repeatedly holding out part of the training data, fitting on the rest, and scoring on the held-out part. The scores from the repeats are averaged (and their spread reported) to give an estimate of generalization error.
  - For cross validation, the folds already play the role of the validation dataset, so no need to split twice.
  - A separate validation set is sometimes still needed, especially with neural networks, or if fitting the model “k” times is computationally expensive

The assumption underneath: independent samples

- **Random k-fold assumes samples are independent and identically distributed:** knowing one tells you nothing about another
- **Geophysical data violate this almost everywhere**
- **A random split then puts near-copies of each test sample into the training set.**
- **The error is silent.** Nothing in the code warns you

In space

adjacent grid cells and nearby stations are near-duplicates

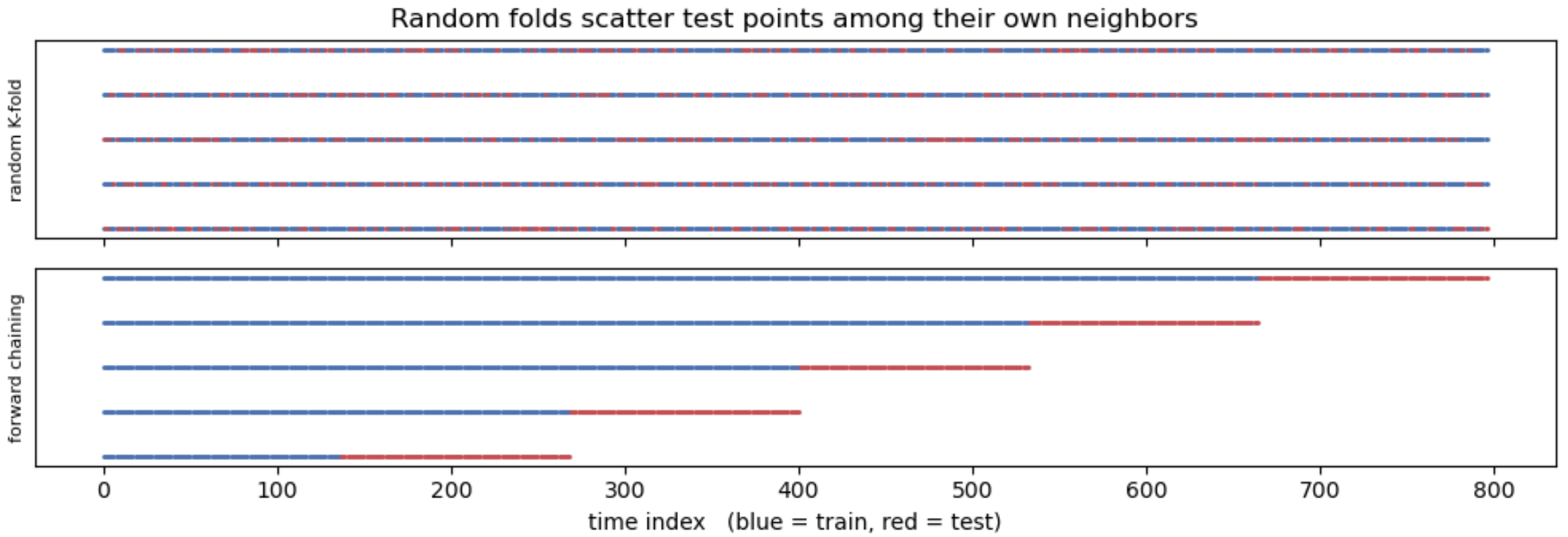
In time

today looks like yesterday; a year of daily rows is far fewer than 365 independent samples

Across the sample

one satellite orbit, one instrument, one model ensemble: shared structure

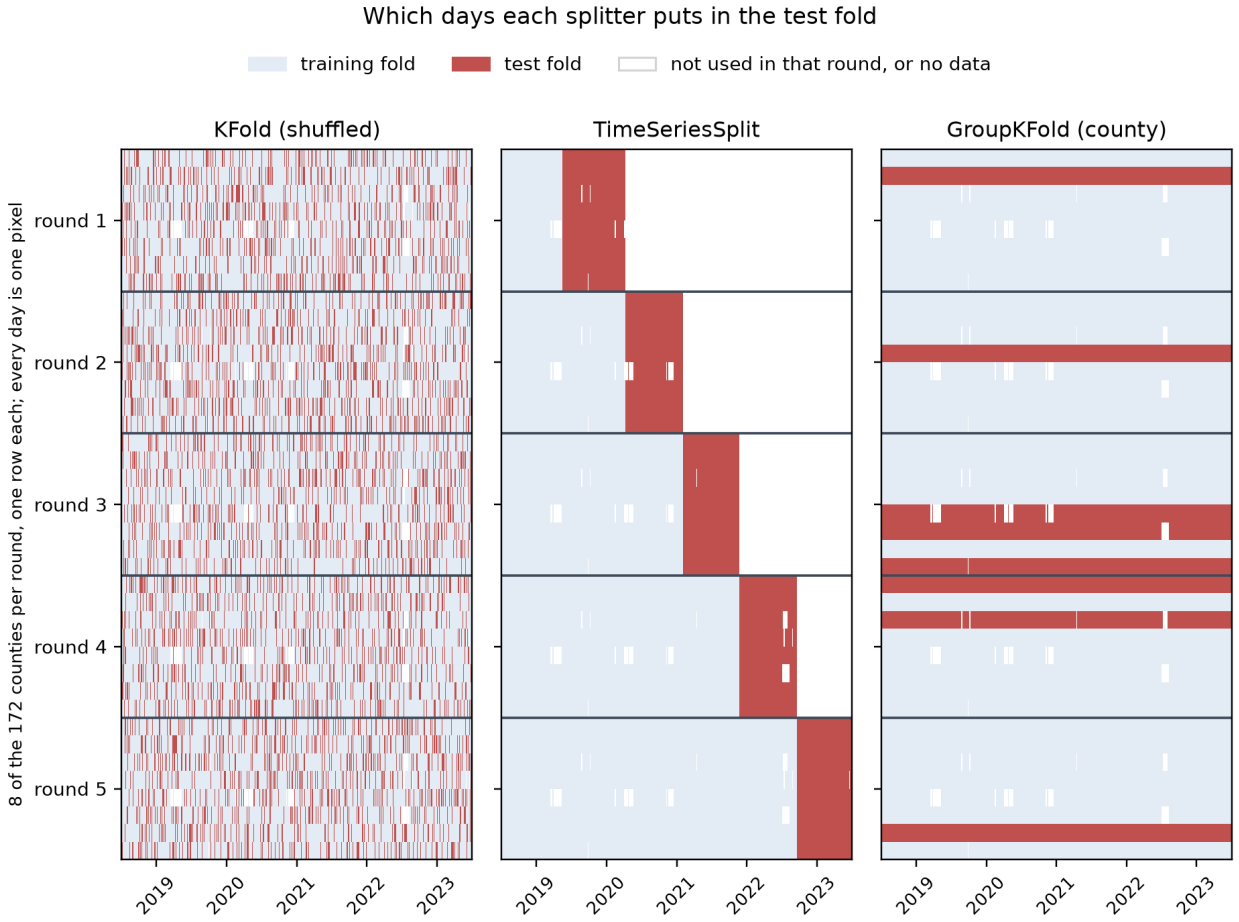
Time: random folds train on the future



Same forest, same data	R^2
Random 5-fold	0.44
Forward chaining (TimeSeriesSplit)	−0.36
Persistence	0.94

- **A slow unobserved mode makes the target unlearnable from the features**
- **Random folds say 0.44:** a model that seems to work
- **Forward chaining says worse than the mean.** That is the truth, and persistence beats everything
- **The random score was not inflated, it was invented,** from the model recognizing its neighbors in time

Three splitters, three questions



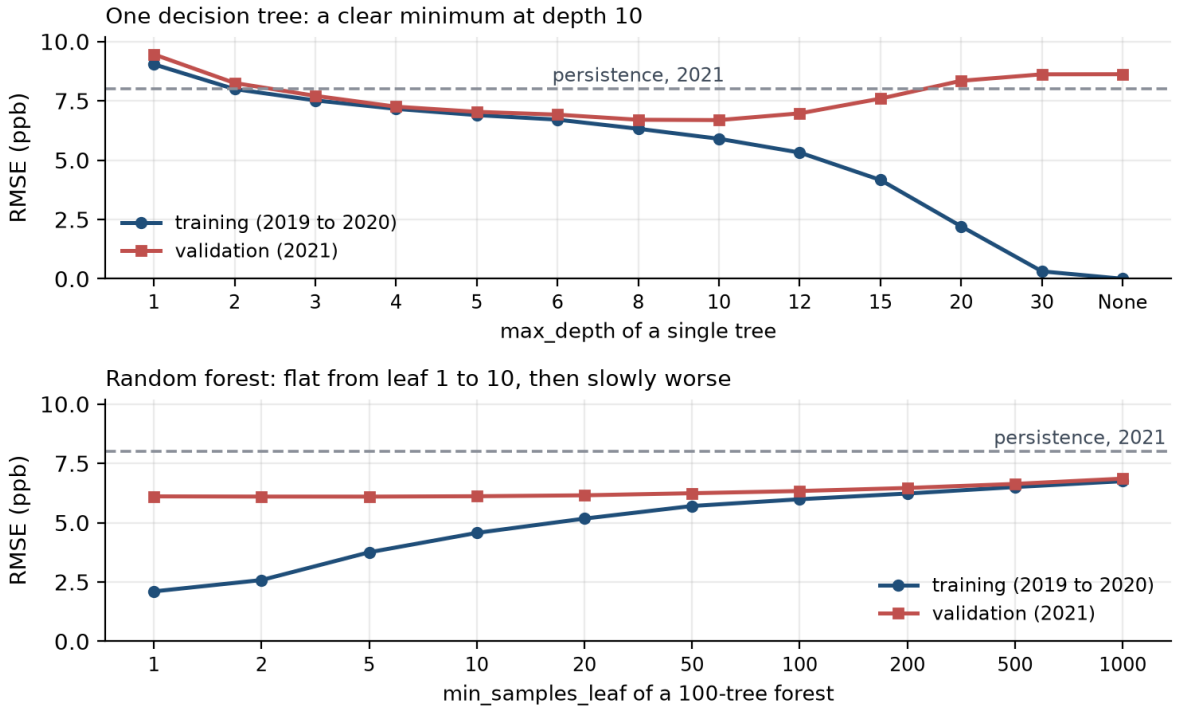
Split, 100-tree forest	RMSE (ppb)	Persistence
A: KFold, shuffled	5.67 ± 0.02	7.97
B: TimeSeriesSplit	6.03 ± 0.22	7.97
C: GroupKFold by county	6.07 ± 0.17	7.97
Test year 2022, trained on 2019 to 2021	5.86	7.95

- **Each splitter asks a different question:** fill in a missing day, forecast next year, or handle a county with no monitor
- **The random split flatters by 0.4 ppb.** It tests each day with its neighbors in the training set. The time and county splits agree with each other and with the test year
- **Same data, same code, same forest.** The split is the question. This model passes all three, which is what makes it deployable

EPA Air Quality System daily maximum 8-hour ozone, county means, 172 counties in 50 states, 2019 to 2023. The figure shows 8 of the 172 counties

Tuning hyperparameters by hold-out validation

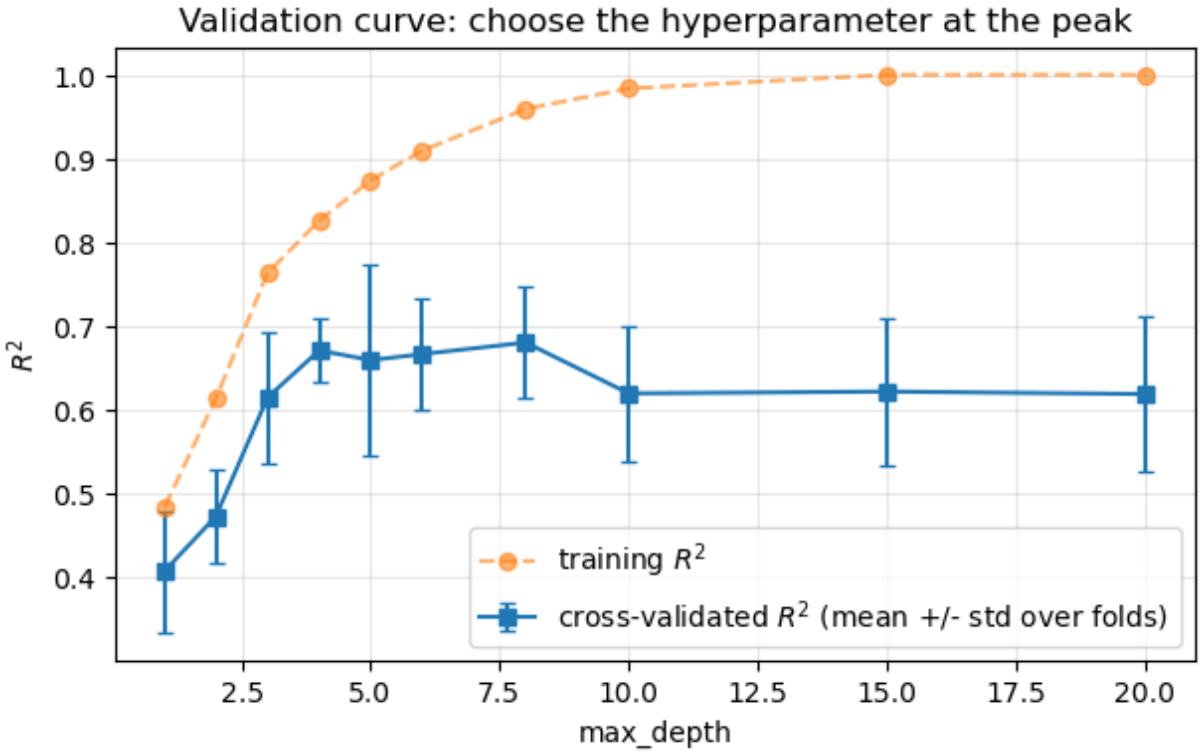
Training	Validation	Test
2019 to 2020	2021: chooses the setting	2022: scored once



- **A hyperparameter is a setting fit does not learn:** depth, leaf size, penalty strength, number of trees
- **The rule:** the data that choose it cannot be the data that score the model. Picking the best test score picks a lucky setting
- **The three-way split:** train, validate to choose, test once at the end
- **The tree's curve bottoms out at depth 10. The forest's is flat from leaf 1 to 10,** which is why Week 5 called forests forgiving. Refit on 2019 to 2021, then one test score each: tree 6.48 ppb, forest 5.86, persistence 7.95

EPA Air Quality System daily maximum 8-hour ozone, county means, 172 counties in 50 states, 2019 to 2023. Training 2019 to 2020, validation 2021, test 2022

Tuning by cross-validation: GridSearchCV




```
grid = {"max_depth": [1, 2, 3, 4, 5, 6, 8, 10, 15, 20]}
search = GridSearchCV(DecisionTreeRegressor(), grid,
                      cv=KFold(5, shuffle=True), scoring="r2")
search.fit(X_train, y_train)          # 10 depths x 5 folds
search.best_params_, search.best_score_, search.score(X_test, y_test)
```

- **What it takes:** a model, a dictionary of candidate values for its settings, a splitter, and a score
- **What it does:** for every combination in the grid, k-fold cross-validation on the training rows. Ten depths and five folds is 50 fits, and never a test row
- **What it keeps:** best_params_, the winning setting. best_score_, its mean fold score. best_estimator_, that model refit on all the training rows. cv_results_, the whole table
- **Two conventions:** scikit-learn maximizes, so errors come back negated. Several settings in one grid means every combination, so three settings with five values each is 125 candidates
- **The curve plateaus between depths 4 and 8.** The error bars say the choice inside the plateau is noise
- **Too many combinations?** RandomizedSearchCV tries a fixed number of random ones. A Pipeline inside the search tunes the preprocessing the same way

After the search: inspect the winner, test once

Randomized search, 16 candidates	depth	leaf size	feature fraction	CV RMSE	± std
rank 1	20	8	0.58	6.11	0.30
rank 2	34	14	0.44	6.12	0.29
rank 3	16	10	0.95	6.16	0.30
rank 4	16	17	0.85	6.16	0.30
rank 16	5	18	0.91	6.84	0.31

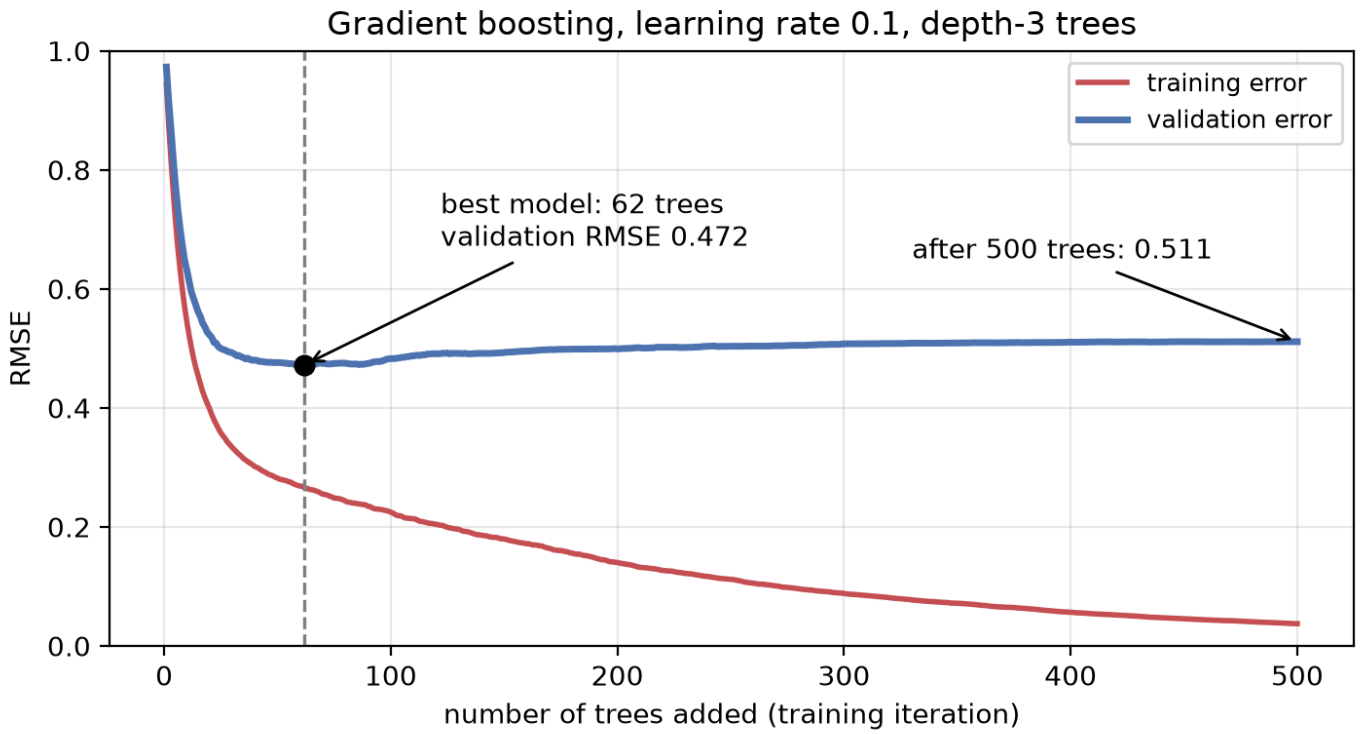
Final forest, 2022, RMSE (ppb) by category	Forest	Persistence
West	5.13	6.31
East and Midwest	6.13	8.65
Texas and south-central	6.83	9.19
December to February	4.82	6.08
June to August	6.88	9.06
Days at or above the 70 ppb standard (376)	12.96	13.23

- **Read cv_results_, not only best_params_.** The top four candidates sit within 0.05 ppb of each other and well inside one standard deviation: a plateau, not a point
- **Inspect best_estimator_.** It is already refit on all the training data. Feature importances, and the errors by category: by region, by season, not only the mean
- **The forest beats persistence in every region and season,** but on the 376 exceedance days it is barely better and 10 ppb low on average. The score alone would not have said so
- **Then one test score, and stop.** do not tweak after measuring the generalization error, or you are overfitting the test set

Géron, Hands-On Machine Learning, 3rd ed., Ch. 2, Randomized Search, Analyzing the Best Models and Their Errors, Evaluate Your System on the Test Set. Top: forest searched with TimeSeriesSplit on 2019 to 2021. Bottom: final forest on 2022. EPA AQS ozone

Extra

Early stopping: the validation set inside training



Reading the figure. The training error never stops falling. The validation error is lowest after 62 trees, 0.472, and is up to 0.511 by 500. Every tree after the 62nd made the model worse at its job.

- **Iterative learners:** gradient descent, boosting, neural networks. Validation error falls, bottoms out, then climbs as the noise gets fitted
- **Stop at the minimum,** or keep going and roll back to the best model. Hinton: a beautiful free lunch
- **Patience:** noisy curves need a wait after the minimum before stopping
- **The validation set is now inside training,** so its score is optimistic. The test set stays separate and is scored once

```
HistGradientBoostingRegressor(early_stopping=True,
    validation_fraction=0.2, n_iter_no_change=20)

keras.callbacks.EarlyStopping(patience=20,
    restore_best_weights=True)      # Weeks 9 and 10
```

Géron, Hands-On Machine Learning, 3rd ed., Ch. 4, Early Stopping. Gradient boosting on the synthetic data from the Week 6 lecture notes, 30 percent held out for validation

Géron's checklist, with this week's steps marked

1. Frame the problem and look at the big picture
2. **Get the data**
3. Explore the data to gain insights
4. Prepare the data to better expose the underlying patterns
5. **Explore many different models and shortlist the best ones**
6. **Fine-tune your models and combine them into a great solution**
7. Present your solution
8. Launch, monitor, and maintain your system

Get the data, item 11. Sample a test set, put it aside, and never look at it (no data snooping!).

Shortlist models, item 2. For each model, use N-fold cross-validation and compute the mean and standard deviation of the performance measure on the N folds.

Fine-tune, item 1. Fine-tune the hyperparameters using cross-validation. Unless there are very few hyperparameter values to explore, prefer random search over grid search.

Fine-tune, item 3. Once you are confident about your final model, measure its performance on the test set. Don't tweak your model after measuring the generalization error.

What the list assumes and ours must add: the folds must match the application. Hold out the future to forecast, the region to map, the station to transfer.

Géron, Hands-On Machine Learning, 3rd ed., Appendix A, Machine Learning Project Checklist. Items quoted from the appendix

Leakage: when the test set reaches the model

Through preprocessing

A scaler, imputer, encoder, or feature selector fitted on all the data has seen the test set. Put every step in a Pipeline so it is refitted inside each training fold.

Ozone, ridge regression: scaler on all rows 6.785 ppb, inside the pipeline 6.785. Invisible here, large for an imputer or encoder, silent either way.

Through a feature

A centered 7-day smoothing window reaches three days into the future and includes the very day being predicted.

Ozone forest: legitimate 5.87 ppb, with the centered feature 5.24, with a trailing window 5.86. Nothing in the code looks wrong.

- **The question to ask of every feature:** would I have this number at the time the prediction is needed?
- **The usual culprits:** centered windows, shift in the wrong direction, future-dated covariates, a column that is the label in disguise
- **If a result seems too good, it probably is.** In this field an unexpectedly high score is more often leakage than a breakthrough

EPA Air Quality System daily maximum 8-hour ozone, county means, 172 counties in 50 states, 2019 to 2023. Trained on 2019 to 2021, scored on 2022

The test score has error bars too

```
from scipy import stats
sq = (final_predictions - y_test) ** 2
np.sqrt(stats.t.interval(0.95, len(sq) - 1,
                         loc=sq.mean(), scale=stats.sem(sq)))
```

Model	Test cases	Test RMSE	95% interval
Géron's housing forest	4,128 districts	$41,424	$39,275 to $43,467
Ozone forest, 2022	47,300 county-days	5.86 ppb	5.81 to 5.91
Persistence, same days	47,300 county-days	7.95 ppb	7.88 to 8.01

Reading the table. The forest's interval sits two ppb below persistence's, and both are a tenth of a ppb wide. The edge is real. But the honest spread is the year-to-year difference, a quarter of a ppb, not the interval.

- **A test score is an estimate from a finite sample,** so it has a standard error. Géron: what if the new model is only 0.1 percent better?
- **Three lines:** a t-interval on the squared errors, then the square root. Width shrinks with the root of the test size
- **For our data the interval is too narrow:** daily errors are autocorrelated and the counties share weather. Score in blocks as a check: the same forest scores 5.86 on 2022 and 6.10 on 2023, outside the interval
- **After heavy tuning, expect the test score a little below the CV score.** Report it. Do not tune again

Géron, Hands-On Machine Learning, 3rd ed., Ch. 2, Evaluate Your System on the Test Set. Ozone rows: EPA Air Quality System daily maximum 8-hour ozone, forest trained on 2019 to 2021

K-fold in scikit-learn: the loop and the shortcuts

```
kf = KFold(n_splits=5, shuffle=True, random_state=0)

scores = []
for train_idx, test_idx in kf.split(X):
    model.fit(X[train_idx], y[train_idx])
    scores.append(model.score(X[test_idx], y[test_idx]))

print(np.mean(scores), np.std(scores))
```

```
scores = cross_val_score(model, X, y, cv=kf, scoring="r2")

detail = cross_validate(model, X, y, cv=kf,
    scoring=["r2", "neg_root_mean_squared_error"],
return_train_score=True)
```

```
rkf = RepeatedKFold(n_splits=5, n_repeats=20, random_state=0)
# 100 fits
scores = cross_val_score(model, X, y, cv=rkf, scoring="r2")
```

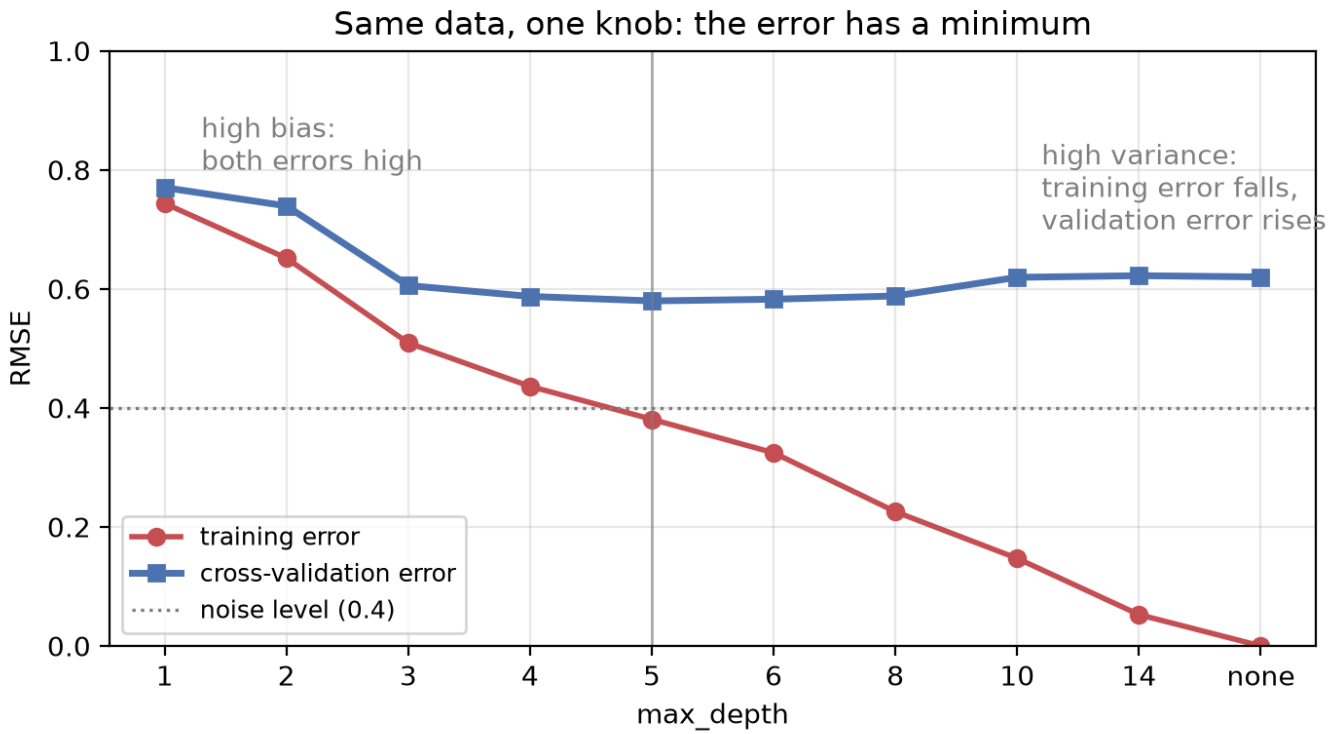
Depth-4 tree, 300 samples	R^2
Five fold scores	0.69, 0.67, 0.58, 0.54, 0.79
5-fold mean ± spread	0.65 ± 0.09
Repeated 20 times (100 fits)	0.67 ± 0.07
100 single hold-outs	0.66 ± 0.07

The splitter is the thing to understand. KFold yields a (train rows, test rows) pair per round. Everything else is bookkeeping.

Error metrics come back negated, because scikit-learn maximizes every score. Flip the sign.

Repeat when the fits are cheap. Five scores give a rough spread; a hundred give a reliable one.

The bias/variance trade-off



Reading the figure. Depths 1 to 3 are the bias regime, depth 8 and beyond the variance regime. The minimum at depth 5 is still well above the noise level of 0.4, because the irreducible part stays.

Bias

wrong assumptions: a model too simple for the data. High bias means underfitting

Variance

sensitivity to the particular training sample: many degrees of freedom fit the noise. High variance means overfitting

Irreducible error

noise in the data itself. Only better data reduces it: fix the sensor, drop the bad retrievals

More complexity: variance up, bias down. Less: the reverse. The validation minimum is where the sum is smallest, and it never drops below the noise.

Géron, Hands-On Machine Learning, 3rd ed., Ch. 4, The Bias/Variance Trade-off. Validation curve on the synthetic data from the Week 6 lecture notes

A random split can misrepresent the data: stratify

Income category	Overall %	Stratified %	Random %	Stratified error	Random error
1 (below $15k)	3.98	4.00	4.24	+0.4 %	+6.4 %
2	31.88	31.88	30.74	0.0 %	−3.6 %
3	35.06	35.05	34.52	0.0 %	−1.5 %
4	17.63	17.64	18.41	+0.0 %	+4.4 %
5 (above $60k)	11.44	11.43	12.09	−0.1 %	+5.6 %

```
train_set, test_set = train_test_split(housing, test_size=0.2,
    stratify=housing["income_cat"], random_state=42)
```

Reading the table. Same data, same 20 percent. The random test set has six percent too many of the poorest and the richest districts, by chance. The stratified one matches the population in every category. Neither model has been fitted yet, and the score is already biased one way or the other.

- **Sampling bias:** a random test set can be unrepresentative by chance, and a small data set makes it likely
- **Stratified sampling:** divide into strata, draw the right number from each. The survey company does not call people at random
- **One argument in scikit-learn,** and StratifiedKFold does the same inside cross-validation. Classifiers get it by default
- **Our strata:** a rare class, a station, a river. Every fold should contain every gauge
- **What it does not fix:** dependence. Stratified daily data are still autocorrelated daily data

Géron, Hands-On Machine Learning, 3rd ed., Ch. 2, Create a Test Set. Table recomputed from the California housing data with Géron's five income categories

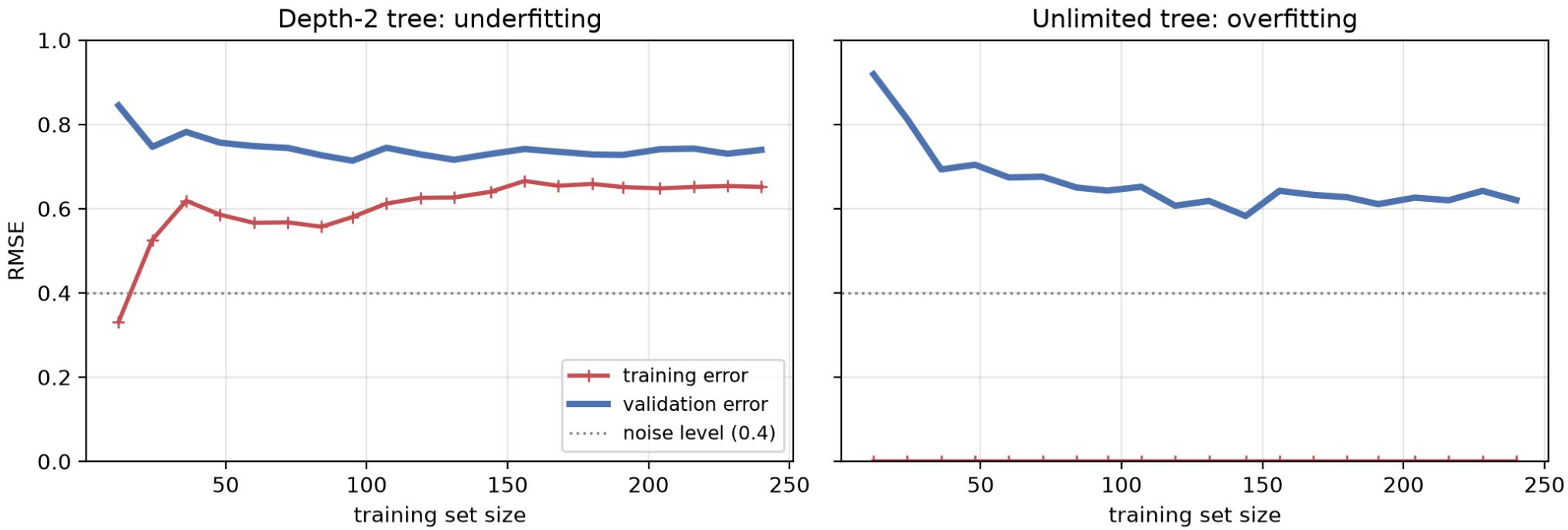
The rules that come up every time

Rule	Why
Shuffle before cutting folds, unless order matters	KFold without shuffle takes contiguous blocks; rows are usually sorted by something
Stratify for classification (StratifiedKFold)	keeps class proportions equal across folds; essential when a class is rare
Put preprocessing inside a Pipeline	a scaler, imputer, or encoder fitted on all the data has seen every test fold
Report mean and spread, not mean and standard error	fold scores share training data and are not independent
Refit on all the data at the end	the k cross-validation models are throwaway; the deployed model is one fit on everything
Flip the sign on error metrics	scikit-learn maximizes, so RMSE arrives as neg_root_mean_squared_error

Every one of these is a form of the same rule: nothing about the test fold may touch the model before it is scored.

Following the Week 6 lecture notes and Géron, Hands-On Machine Learning, 3rd ed., Ch. 2

Learning curves: too simple, or too flexible?



Underfitting, left: both curves plateau high and close together. More data will not help. Use a more flexible model or better features.

Overfitting, right: training error near zero and a gap to the validation curve. More data narrows the gap. So does a simpler model.

Géron, Hands-On Machine Learning, 3rd ed., Ch. 4, Learning Curves, with learning_curve(). Data: the synthetic set from the Week 6 lecture notes, noise standard deviation 0.4